Multi-Core: Global Scheduling

Md Mostafizur Rahman
Electronic Engineering
Hochschule Hamm-Lippstadt (HSHL)
Lippstadt, Germany
md-mostafizur.rahman@stud.hshl.de

***Abstract*—Multicore processors increase the computing capacity available to embedded and hard real-time systems, but additional cores alone do not guarantee that deadlines are met. This report studies Global Earliest-Deadline-First (G-EDF) scheduling with emphasis on the interaction between scheduling theory and operating-system implementation. The mathematical task model, G-EDF and Partitioned EDF (P-EDF) algorithms, and the preemption-centric worst-case execution-time (WCET) inflation method are summarized from the reference study by Gracioli et al. An experimental transfer study is then performed on the same AMD Ryzen 7 7435HS platform under Windows 11 and Ubuntu Linux. A CPU-bound task, with no load and with fifteen background workers. The measured WCET rises from 437 to 819 ms on Windows and from 309 to 458 ms on Linux. Linux `perf` also reports 18,220 context switches, 13 CPU migrations and 7.14 million cache misses for the complete benchmark. Finally, the measured Linux values are encoded into a UPPAAL two-core model. The verifier confirms a safe case and outputs a counterexample if the loaded task's 458ms WCET exceeds a 400ms deadline. The results show, without claiming a complete EPOS–LITMUS^{RT} [1] reproduction, that the operating-system interference can affect the schedulability.**

***Index Terms*—G-EDF, P-EDF, real-time operating systems, WCET, EPOS, LITMUS^{RT}, UPPAAL, multicore scheduling**

I. Introduction

Multicore processors are increasingly used for the integration of functions that were previously distributed over multiplewere electronic controldistributed over multiple electronic control units.units. Representative examples include automotive driver assistance, braking control, sensor fusion and machine-vision pipelines. Functional correctness is not enough in such systems: a job that produces the correct output after its deadline is considered to be incorrect. Therefore, the design of hard real-time (HRT) systems consists of a scheduling policy and an implementation with bounded timing costs.

Two common multicore approaches are global and partitioned scheduling. In G-EDF, all ready jobs are in one priority queue, and the m cores run the m jobs with the smallest absolute deadlines. Jobs could move after preemption. P-EDF dispatches tasks to cores in an off-line manner and each core executes a local EDF scheduler. The global approach offers flexible load sharing while the partitioned approach avoids migration but poses an NP-hard bin-packing problem.

A central finding of Gracioli et al. is that this algorithmic comparison cannot be separated from the operating system (OS) [1]. Context switch, scheduling decision, task release, timer tick, inter-processor interrupt (IPI) and cache related preemption/migration delay (CPMD) further inflate the effective execution time used by schedulability tests. The overhead of their global and partitioned EDF implementation in the purpose-built EPOS real-time operating system is much lower than the LITMUS^{RT} real-time Linux infrastructure. Idealized theory often favors partitioning, but in some task-set regions, low-overhead G-EDF on EPOS outperforms P-EDF on LITMUS^{RT}.

The report provides a concise end-to-end study of (i) mathematical foundations and formal algorithm descriptions, (ii) runtime and implementation tradeoffs, (iii) a reproducible WCET experiment on Windows/Linux, and (iv) a UPPAAL timing model derived from measured values. The local study is clearly a qualitative transfer experiment rather than a reproduction of the original RTOS comparison.

II. Mathematical Model and Scheduling Algorithms

A. Periodic Task Model

Let $\tau = \{T_1, \ldots, T_n\}$ be a set of periodic tasks scheduled on m identical cores $\{P_1, \ldots, P_m\}$. Task T_i is described by its execution time e_i, period p_i, and relative deadline d_i. The implicit-deadline model used in the reference work assumes

$$d_i = p_i, \qquad u_i = \frac{e_i}{p_i}, \qquad U = \sum_{i=1}^{n} u_i, \tag{1}$$

where u_i is task utilization and U is total utilization. The j-th job of T_i released at r_i^j has absolute deadline $a_i^j = r_i^j + d_i$. An HRT task set is schedulable if every released job completes no later than its absolute deadline. For uniprocessor EDF with implicit deadlines, $U \leq 1$ is necessary and sufficient [2]. Multiprocessor G-EDF instead relies on sufficient tests that may reject feasible task sets [4], [5].

B. Global EDF

G-EDF maintains one ready queue ordered by a_i^j. At each scheduling event we can select m earliest deadline jobs. released, the running job with the furthest deadline may be preempted. job is running on another core, an IPI requests remote rescheduling. inserted back into the global queue and may later resume on a different core leading to migration and possible cache-affinity loss. head is $O(1)$. Algorithm 1 describes the online behavior.

Algorithm 1 G-EDF(τ, m)

$Q \leftarrow$ global queue ordered by `abs_deadline`
on release of J_i at time t:
  J_i.`abs_deadline` $\leftarrow t + d_i$; Q.insert(J_i) $\{O(n)\}$
  $J_{low} \leftarrow$ running job with latest deadline
if J_i.`abs_deadline` $< J_{low}$.`abs_deadline` **then**
  preempt(J_{low}); Q.insert(J_{low})
  IPI_send(core_of(J_{low})) {remote reschedule}
end if
on core free: dispatch(Q.remove_head()) $\{O(1)\}$

Algorithm 2 P-EDF-FFD(τ, m)

sort τ by u_i descending; $cap[1..m] \leftarrow 1.0$
for each $T_i \in \tau$ **do**
  find smallest k with $cap[k] \geq u_i$
  if none exists **then**
    return INFEASIBLE
  end if
  assign $T_i \rightarrow P_k$; $cap[k] \leftarrow cap[k] - u_i$
end for
verify (2); each P_k runs local EDF {no migration}

C. Partitioned EDF

P-EDF separates scheduling into an offline allocation phase and online local EDF execution. Heuristic such as First-Fit Decreasing (FFD) sorts tasks in decreasing order of utilization, and maps each task to the first core with sufficient remaining capacity (Algorithm 2). core will schedule only its assigned tasks using EDF. The per core tests are:

$$\sum_{T_i \in P_k} u_i \leq 1, \qquad k = 1, \ldots, m. \tag{2}$$

Partitioning requires no migration or cross-core IPI. to bin packing and NP-hard. example, five tasks with $u_i = 0.51$ cannot be scheduled on four cores even if $U = 2.55 < 4$: a single core can host at most one such heavy task. Table I compares the two approaches.

III. Runtime Overhead and OS Implementation

A. WCET Inflation

The execution time used for analysis must include implementation costs. The reference study uses preemption-centric accounting [3]. Let c^{xs}, c^{scd}, c^{rel}, c^{tck}, c^{ipi}, and c^{pd} denote context-switch, scheduling, release, tick, IPI, and CPMD costs. With timer period Q,

$$u_0^{tck} = \frac{c^{tck} + c^{rel}}{Q}, \tag{3}$$

$$c^{pre} = \frac{c^{tck} + c^{rel}}{1 - u_0^{tck}}, \tag{4}$$

$$e_i' = \frac{e_i + 2(c^{scd} + c^{xs}) + c^{pd}}{1 - u_0^{tck}} + 2c^{pre} + c^{ipi}. \tag{5}$$

The schedulability tests use the inflated value e_i' instead of e_i. The two scheduling and context-switch terms account for release/completion boundaries. CPMD captures lost cache affinity. IPI latency is modeled as computation because the considered G-EDF tests do not model self-suspension. The relationship between OS design and scheduling feasibility is formalized in Equation (5).

TABLE I
G-EDF and P-EDF Tradeoffs

Property	G-EDF	P-EDF
Queues	one global queue	one queue per core
Migration	allowed	prohibited
IPI	possible remote reschedule	not needed for migration
Queue cost	$O(n)$ ordered insertion	about $O(n/m)$ per core
Strength	dynamic load sharing	predictable locality
Weakness	CPMD, pessimistic tests	NP-hard allocation

TABLE II
Worst-Case Overhead: EPOS vs. LITMUS^{RT} [1]

Overhead	EPOS	LITMUS^{RT}	Factor
Context switch	0.30 μs	$\leq$ 29.56 μs	76×
IPI latency	$\sim$0.30 μs	$\leq$ 4.5 μs	15×
Scheduling (G-EDF)	$O(n)$, stable	unpredictable	7×
Scheduling (P-EDF)	$O(n/m)$, stable	unpredictable	21×
Tick counting	0.248 μs const.	grows with n	84×

B. EPOS and LITMUS^{RT}

EPOS uses compile-time metaprogramming, library-mode execution, and hardware mediators inlined into the application code to minimize runtime overhead. Compile-time scheduling criterion without a runtime policy branch selects either one global queue or per-core queues. The distributed alarm handler maintains per-core event lists, and the scheduler can send to IPI when a newly released global job must replace the lowest-priority running job. LITMUS^{RT} is a Linux extension for real-time exposing scheduling plugins and tracing support; its flexibility inherit also Linux complexity and variability. Table II presents the worst-case overhead measured in [1] on an Intel i7-2600 (4 cores / 8 logical, 3.4 GHz).

EPOS G-EDF overhead grows after about 75 threads, since its ordered list has $O(n)$ operations; P-EDF is steadier, as each queue has only a portion of the tasks. The key conclusion is not that G-EDF is always better than. Instead, an implementation with higher theoretical bounds may perform worse than another implementation with sufficiently lower overheads for the underlying OS. structure, interrupt architecture and cache behavior must be evaluated together.

IV. Reference Evaluation and Scalability

A. Schedulability Experiments

The reference work considers both idealized scheduling theory and measured implementation overhead. considered. For P-EDF, First Fit, Best Fit and Worst Fit are followed by per-core EDF tests on decreasing partitioning, Synthetic task sets are defined by varying period length, utilization range and utilization shape (uniform or bimodal), leading to eighteen

workload families. ideal reference is to 100 processors, but OS overhead is measured on an eight-logical-processor platform. For each utilization bound, the schedulability ratio is the proportion of generated task sets that satisfy a given test. G-EDF is more vulnerable since migration is permitted while P-EDF maintains core affinity after allocation.necessarily infeasible, because the multiprocessor tests are necessary but not sufficient; G-EDF bounds are especially pessimistic for heavy or bimodal task sets, and P-EDF can reject a feasible set when its bin-packing heuristic fails.

The general within-platform result is that P-EDF is equal or better than G-EDF for the examined HRT scenarios. This does not conflict with the reported cross-platform reversal: in selected light-utilization, short-period cases G-EDF with EPOS overhead outperforms P-EDF with LITMUS$^{\mathrm{RT}}$ overhead. First comparison isolates scheduling policy under the same implementation assumptions, the second comparison shows that the quality of the OS can dominate the policy advantage.

B. CPMD and Weighted Schedulability

Cache-related preemption and migration delay depends on the working-set size, cache hierarchy, and whether a resumed job returns to the same core. vulnerable since migration is permitted while P-EDF maintains core affinity after allocation. The reference work estimates CPMD with hardware performance counters and controlled cache replacement. To report results across utilization levels it uses a weighted schedulability metric

$$W(D_c) = \frac{\sum_{U \in \mathcal{Q}} U \, S(U, D_c)}{\sum_{U \in \mathcal{Q}} U}, \tag{6}$$

where $S(U, D_c)$ is the schedulability ratio at utilization cap U and CPMD bound D_c. The weighting by U guarantees that low and high load task sets are not treated as equally informative. With increasing CPMD, the advantage of global load balancing is increasingly eaten up by the cache refill costs, and thus G-EDF and P-EDF performance can become similar.

Data structures and interrupt organization also contribute to scalability. Locking and cache coherence would matter for large core counts, but using a heap instead of an ordered list would bring queue operations down to $O(\log n)$. P-EDF spreads queue operations and contention across cores, but it continues to pay the price of offline allocation, and the fragmentation of spare capacity. observations motivated clustered or semi-partitioned schedulers that limit migration to groups of cores and aim for a middle ground between global flexibility and partitioned locality [6].

V. EXPERIMENTAL METHOD

A. Platform and Benchmark

The transfer experiment was carried out on a Lenovo LOQ 15ARP9, which features an AMD Ryzen 7 7435HS with eight physical cores, sixteen logical processors enabled by simultaneous multithreading, and a shared 16 MB L3 cache.

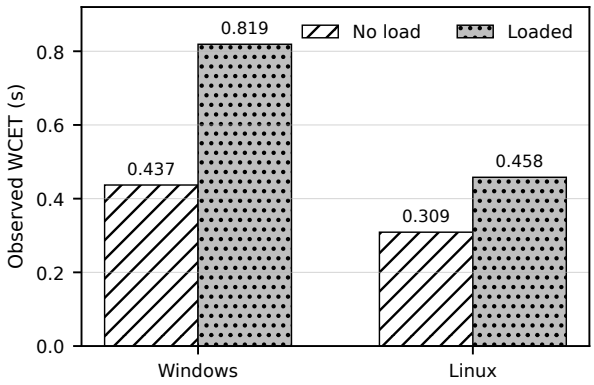

Fig. 1. Sampled WCET with and without fifteen background workers on identical hardware.

same benchmark was executed on Windows 11 and Ubuntu Linux.

Target task runs the integer loop $x \leftarrow x + i^2$ for five million iterations. memory communication is performed. first phase the task is timed ten times without generated load. second phase, fifteen worker processes repeatedly execute the same loop, timing the target task ten more times. phase the sampled WCET is taken to be the maximum observation. an empirical maximum and not an analytical HRT bound.

On Linux, the full program was run with `perf stat` to count context switches, CPU migrations, cache references, cache misses, cycles, and instructions. These counters count the target process and all background workers; they are indicators of system-level interference, not isolated costs for c^{xs}, c^{scd}, or c^{pd}.

B. Benchmark Implementation and Reproducibility

The benchmark is purposely small so that the load and timing relationship is obvious. done in separate processes to avoid Python's global interpreter lock. benchmark is executed on Linux with temporary access to counters enabled via `perf_event_paranoid` and restored after `perf stat`. To reproduce the results, the interpreter, OS, processor topology, number of workers, number of iterations and runs, and power-management state must be reported. As the initial Linux no-load observations were slower than subsequent runs, additional measurements should perform explicit warm-up iterations and more repetitions prior to selecting a high percentile or maximum.

C. Results

Fig. 1 and Table III compare the maximum observations. On Windows, the no-load WCET was 437 ms and the loaded WCET was 819 ms, an increase of 382 ms or 87.3%. On Linux, the `perf` run produced 309 ms without load and 458 ms with load, an increase of 149 ms or 48.5%. A separate plain Linux run gave 301 ms and 478 ms, consistent with the same qualitative result.

TABLE III
LOCAL EXPERIMENTAL RESULTS

OS	No load	Loaded	Increase	Relative
Windows 11	437 ms	819 ms	382 ms	87.3%
Ubuntu Linux	309 ms	458 ms	149 ms	48.5%

TABLE IV
LINUX PERF COUNTERS (FULL PROGRAM)

Counter	Value
Context switches	18,220
CPU migrations	13
Cache references	3.484×10^9
Cache misses (rate)	7.135×10^6 (0.20%)
Cycles	3.761×10^{11}
Instructions (IPC)	1.014×10^{12} (2.70)

Listed in the Table IV.,the Linux scheduler as a whole preserved placement while context switch count indicates significant scheduling activity. the low miss rate and high IPC indicate that the benchmark was still largely compute bound. measured slowdown should be described as a combined WCET inflation due to CPU sharing, scheduling, context switching, SMT contention and cache effects, but not as a direct measurement of one individual RTOS overhead source.

Our experiment does not provide a universal ranking of Windows/Linux. It shows that the OS environment alters the observed effective WCET for this workload and hardware which is a qualitative analogue of the EPOS–LITMUS$^{\text{RT}}$ result.

VI. G-EDF APPLICATION AND UPPAAL VERIFICATION

A. Example of Two-Core Application

Two jobs released at time $t = 0$ on two cores. Both are put in one global queue EDF. Both jobs are dispatched immediately, without waiting or preemption, as there are two cores free. Let $T_1 = 309\,\text{ms}$ and $T_2 = 458\,\text{ms}$, using the measurements from Linux. G-EDF schedules the available jobs correctly and exploits both cores but the scheduler is not able to compensate for the case when the effective WCET exceeds the deadline. miss scenario, both deadlines are $400\,ms$. The scheduling decision is the same in both scenarios, but the loaded task misses, because $458 > 400$ (Fig. 2).

Example shows the importance of overhead. available jobs correctly and exploits both cores but the scheduler is not able to compensate for the case when the effective WCET exceeds the deadline. Adding more processing resources does not eliminate a per-job timing violation.

B. UPPAAL model

The UPPAAL model has two independent cases in one XML system. Each case is two tasks and two cores with integer millisecond clocks. In the safe automaton, T_1 finishes at 309 ms before the deadline of 400 ms and T_2 finishes at 458 ms before the deadline of 600 ms. miss automaton, T_2 has

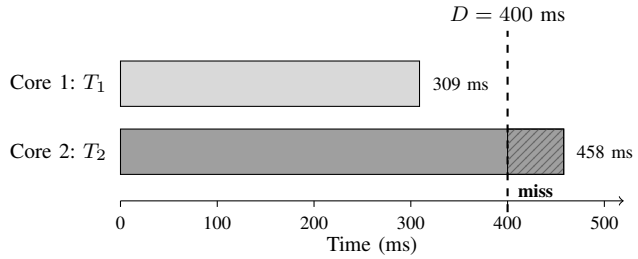

Fig. 2. Two-core G-EDF execution in the Linux miss scenario. Both jobs start at $t = 0$; the hatched region shows T_2 executing past the 400 ms deadline.

TABLE V
UPPAAL QUERIES AND VERDICTS (COMBINED MODEL)

Query	Verdict
$A[]$ ¬deadlock	true
$E\langle\rangle$ Safe.RunningBoth	true
$E\langle\rangle$ safe_core1 $= T_1 \wedge$ safe_core2 $= T_2$	true
$E\langle\rangle$ Safe.DoneAll	true
$A[]$ ¬safe_deadline_miss	true
$E\langle\rangle$ Miss.RunningBoth	true
$E\langle\rangle$ Miss.Missed	true
$E\langle\rangle$ miss_deadline_miss	true
$A[]$ ¬miss_deadline_miss	**false**

a 400 ms deadline, its clock reaches the deadline with 58 ms of execution remaining, and the automaton enters `Missed`. The invariant of the loaded location uses the *deadline* rather than the execution time, asthe deadline is the tighter bound: time cannot be more than 400 ms, and therefore the completion edge guarded by $y_2 = 458$ is structurally unreachable, ie, the formal expression of "the task cannot finish in time.Terminal locations are provided with self-loops to prevent artificial deadlock. nine verification queries and their verdicts are summarized in Table V.

The loaded execution of 458 ms does not meet the 400 ms deadline in *any* execution path, which is the expected final falsified property and provides the counterexample. The model thus yields a formal consequence of the timing values measured. It does not model arbitrary releases, migration or IPI behavior, nor does it verify the full EPOS or LITMUS$^{\text{RT}}$ scheduler.

VII. INTERPRETATION AND LIMITATIONS OF SCHEDULABILITY

The deadline test directly reflects the measurements. Under Windows, $437 < 700\,\text{ms}$ is safe while $819 > 700\,\text{ms}$ misses. Under Linux, $309 < 400\,\text{ms}$ is safe while $458 > 400\,\text{ms}$ misses. A utilization-only example illustrates why capacity and deadlines must not be confused. If two identical Linux loaded tasks have period 1000 ms, then

$$U = 2\left(\frac{458}{1000}\right) = 0.916 < 1. \tag{7}$$

A single-core implicit-deadline utilization test (deadlines equal to periods) would accept this synthetic task set. separate

400 ms demonstration deadline is shorter than the 1000 ms period and is missed by a 458 ms job. The apparent contradiction disappears when the models have different deadline assumptions, with $d_i < p_i$ the problem becomes *constrained-deadline* and requires deadline-aware response-time analysis rather than only $U \leq 1$. Correct analysis must preserve real relationship between execution time, period and deadline.

Conclusions are bounded by six limitations. First, Windows and plain Ubuntu are general purpose systems and they are neither EPOS nor LITMUS^{RT}. Second, the maximum of ten runs is a sampled WCET and might miss rare events. Third, perf counters are summed over the entire process group, and hence do not provide the individual context-switch, tick, IPI and CPMD values. Fourth, the UPPAAL model checks some timing cases, but not a generic multicore EDF implementation. Fifth, the benchmark is an arithmetic loop which is compute bound and has a small working set; an I/O bound or memory bound application may exhibit different cache and migration behaviors. Sixth, the implicit-deadline periodic model does not account for sporadic arrivals, bounded deadlines, shared-resource blocking, and mixed-criticality transitions. The library mode of EPOS also trades isolation for speed due to the application and OS sharing an address space, which is a concern for safety-certified deployment. A stronger approach would be to use kernel tracing or an RTOS testbed, control core affinity and real-time priority, measure each overhead source separately, and model multiple periodic releases with explicit preemption, migration, and resource blocking.

Despite these limitations, the evidence chain is coherent: the reference paper identifies WCET inflation sources, the local benchmark measures aggregate WCET inflation on real hardware, Linux counters reveal scheduling and cache activity, and UPPAAL verifies the resulting deadline consequence.

A. Practical Design Implications

Several engineering guidelines follow. First, the choice of scheduler should not be based on theoretical processor utilization alone: the cost of the actual run queue, timer implementation, interrupt path and cache hierarchy should be included in the analysis. Second, execution time should be measured under idle, representative and stress conditions, explicitly reporting CPU affinity, real-time priority and SMT — an isolated benchmark is optimistic, full saturation might be overly pessimistic.

Third, the task model should be internally consistent: if the demonstration uses a deadline smaller than the period (eg, the 400 ms miss case), thenthe problem is constrained deadline and requires deadline-aware response-time analysis, not simply $U \leq 1$. Finally, measurement and formal verification are complementary: measurement provides realistic parameters, but cannot exhaustively cover every path; timed automata cover every behavior admitted by the model, but are only as realistic as their parameters.

VIII. CONCLUSION

This report considers global scheduling on multicores from the mathematical, implementation, experimental and formal points of view. G-EDF provides a single global deadline-ordered queue that is shared and can be used flexibly by all cores, but migration, IPIs, and a larger shared queue introduce overhead. P-EDF improves locality and per-core scalability by requiring NP-hard offline partitioning. The Comparison of EPOS and LITMUS^{RT} illustrates that the OS architecture can have a substantial impact on the practical outcome of the tradeoff.

The local transfer experiment is consistent with the same qualitative conclusion. The WCET increase was 87.3% on the Windows platform and 48.5% on the Linux platform under background CPU load. The Linux `perf` data confirmed that there were a lot of scheduling activities and the UPPAAL model formally proved that an effective WCET of 458 ms does not meet a 400 ms deadline, even with two cores and G-EDF dispatching both jobs immediately. Therefore, schedulability must be analyzed with realistic effective WCET, not just ideal algorithmic assumptions and processor count.

REFERENCES

[1] G. Gracioli, A. A. Fröhlich, R. Pellizzoni, and S. Fischmeister, "Implementation and evaluation of global and partitioned scheduling in a real-time OS," *Real-Time Systems*, vol. 49, pp. 669–714, 2013.

[2] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[3] B. B. Brandenburg, "Scheduling and locking in multiprocessor real-time operating systems," Ph.D. dissertation, Univ. North Carolina at Chapel Hill, 2011.

[4] J. Goossens, S. Funk, and S. Baruah, "Priority-driven scheduling of periodic task systems on multiprocessors," *Real-Time Systems*, vol. 25, pp. 187–205, 2003.

[5] M. Bertogna and M. Cirinei, "Response-time analysis for globally scheduled symmetric multiprocessor platforms," in *Proc. IEEE Real-Time Systems Symp.*, 2007, pp. 149–160.

[6] A. Bastoni, B. B. Brandenburg, and J. H. Anderson, "An empirical comparison of global, partitioned, and clustered multiprocessor EDF schedulers," in *Proc. IEEE Real-Time Systems Symp.*, 2010, pp. 14–24.